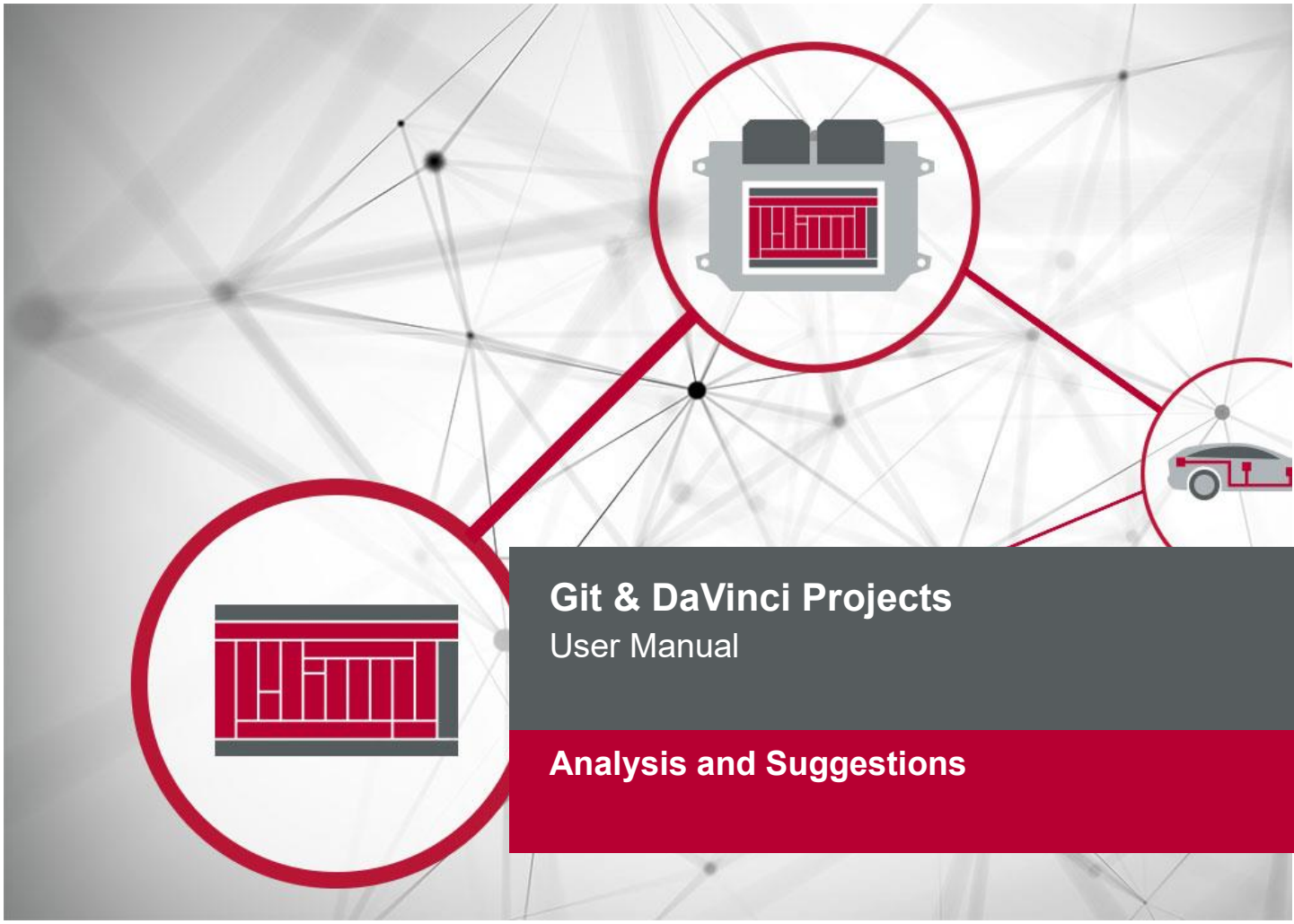
VECTOR



Git & DaVinci Projects

User Manual

Analysis and Suggestions

Daniel Gamiz Antunez

V1.1.0 | Released

Document Information

History

Author	Date	Version	Remarks
Daniel Gamiz Antunez	2020-09-10	1.0.0	Initial Version
Daniel Gamiz Antunez	2020-11-20	1.0.1	Clarified that this document also can be used for DaVinci Developer Projects
Daniel Gamiz	2022-12-12	1.1	Remake of the document



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

© 2022 Vector Informatik GmbH

Contents

1 Introduction 5

2 Why use the DaVinci Merge? 7

3 Git and DaVinci Project Merge 9
3.1 Overview 9
3.2 Git & DaVinci Merge Steps 10
3.2.1 Getting local copies of files to be merged 10
3.2.2 Git Branch Merge 11
3.2.3 Discard DaVinci Project Merge 11
3.2.4 DaVinci Project Merge with Comparison Mode 11
3.2.5 Test your Merge (CI/CT) 15
3.2.6 Merge Completion 15
3.2.7 Push your changes 15

4 Glossary and Abbreviations 16
4.1 Glossary 16
4.2 Abbreviations 16

5 Contact 17

Tables

Table 5-1 Glossary 16
Table 5-2 Abbreviations 16

1 Introduction

Git is usually used because it offers the following advantages:

- It is fast.
- The distributed model allows offline work.
- It can maintain history of changes.
- Branching and merging are relatively easy.
- It includes an Index/Staging area that allow commits to have logically grouped changes.

While Git and DaVinci projects can work together, Git's standard workflow is not recommended for an efficient merge. There are two main reasons for it:

First, Git´s textual merge can be time consuming.

- The nature of an EcuC Project may result in data that is not necessarily relevant but still causes conflict/differences (See 2 Why use the DaVinci Merge?Why use the DaVinci Merge?).
- It may be difficult to know if a textual merge follows the rules and restrictions of AUTOSAR.
- The inner workings of DaVinci Projects include some auto-solving actions which may lead to unexpected merge-conflicts.

For a safe merge, DaVinci Tools have its own model-based merge tool which uses the model itself as a reference instead of individual ARXML fragments and uses simplified representations of the model for merge decisions. This makes decision-making easier, and possible for different sources regardless of its format.

Secondly, juggling between both Git and DaVinci merges can be unintuitive and uncomfortable.

For that reason, **DaVinci Team** was created. DaVinci Team is a tool developed by Vector to be used when collaboration with the AUTOSAR Model is intended. DaVinci Team provides plugins for a smooth Git interaction and is Vector´s official recommendation.

If you would like to learn more about DaVinci Team, click here.



Note
DaVinci Team is our recommended tool for collaboration of AUTOSAR models in combination with Git.

The aim of this document is to explain the Legacy Method of merging when using Git. It includes:

- The challenges of classic Git textual merging on DaVinci Projects, or why should we use the DaVinci Merge.

- How to combine Git with a DaVinci Merge.
- Some tips and tricks which can lead to a smoother experience.

In this document, it is assumed that the reader either already has some knowledge of Git or is willing to read the Git documentation to learn about its commands.

2 Why use the DaVinci Merge?

The workspaces of DaVinci Tools mainly consist of XML files and technically quite easy to version with Git. However, there is one main challenge that result from using the Git textual merge:

- Correctness of merge decision during merge cannot be evaluated on model level
  - The tool´s model is not available through the merge, an additional text to model step would be required to achieve this. But, even if done as a further step, finding if the merge resulted in a valid model is available late in the process.

Besides the reason shown above, the specific **inner workings of DaVinci Tools** may result in changes which cause textual merge conflicts even if they are not relevant in a broader sense. Some examples are written below.

Line Endings

DaVinci Configurator currently does not consistently consider the platform-specific End of Line (EOL) control characters when saving. For example, the .dpa file is always saved with Line Feed (LF) on Windows, instead of a Carriage Return, Line Feed (CRLF).

With Git this leads to warnings if the EOL conversion is switched on. Some XML schemes that are supplied in developer projects use both CRLF and LF in one and the same file.

UUID Diffs

During the project update, elements may get new UUIDs even though they have not changed. This is due to the way in which the project update is carried out.

From the user's point of view, however, these diffs are unnecessary and not only complicate the textual merge but are also displayed as diffs in the history of the VCS. The problem affects both the DaVinci Developer and the DaVinci Configurator.

Project Update

A project update could also change the .dvg files for a project with a Developer Workspace, which contains the diagram information for DaVinci Developer. These diffs are not relevant for the actual project update, but they do make the textual merge more difficult.

Conclusion

Despite various merge strategies, textual merging using Git could work, but it is not optimal for DaVinci projects. Even if the difficulties mentioned above are resolved with diffs and ARXML files are merged syntactically correctly (i.e., a well-formed XML is created from them), this unfortunately does not mean the containing data is semantically correct.

In the end it could be said that merging an EcuC configuration is only half of the work. If the configuration changes too much, the corresponding service component should be regenerated. In addition, many other scenarios are conceivable in which, for example, **auto-**

solving actions of the DaVinci Tools could also influence the merge result. Git alone cannot offer a solution to all of this.

For the reasons mentioned **it is essential to carry out the merge using DaVinci Configurator and/or DaVinci Developer**, which has the following advantages:

- Merge on model level possible instead of individual ARXML fragments – this guarantees consistency.
- Merge decisions based on simplified representations of the model (easier to understand what are merging).
- AUTOSAR models from other sources or formats can be imported and stored.

3 Git and DaVinci Project Merge

3.1 Overview

When merging branches, a workflow like this could be expected:

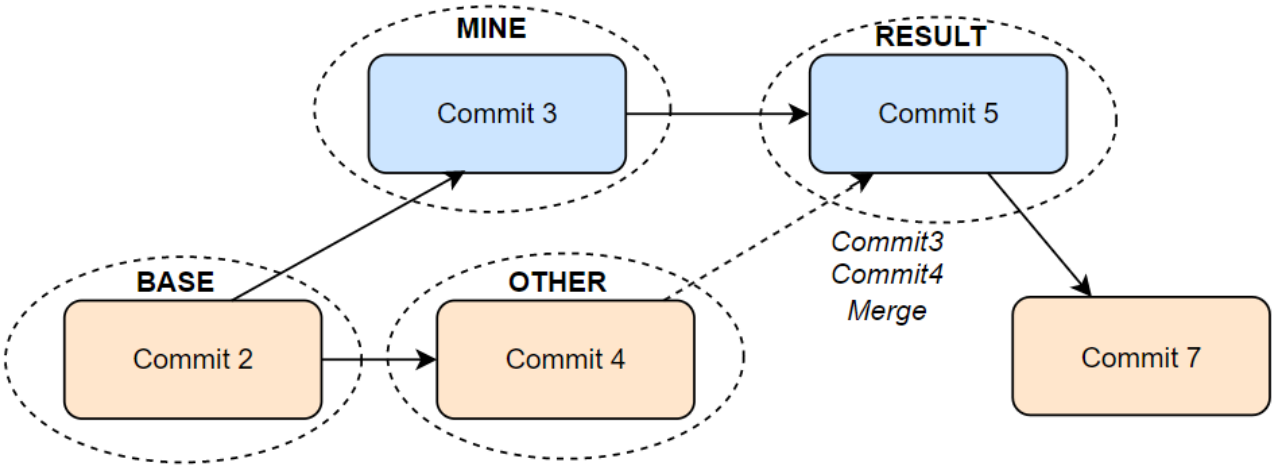


Figure 3-1 Merge workflow example

The usual workflow uses a three-way merge, which means three full projects should exist: BASE, MINE and OTHER. The three-way merge makes the merge proposal based on a common ancestor; this is called BASE. A three-way merge allows the use of the auto-merge button, which will automatically merge all non-conflicting differences.

Overall, short-lived feature branches are highly recommended as well as CI/CT (Continuous Integration / Continuous Testing) pipelines to confirm that the resulting merge is valid before merging feature branches.

For merge strategies, see the **DaVinci Team & Platform Support** document.

This is a quick overview of how the merge steps look like:

1. Getting local copies of the DaVinci Projects to be merged
2. Git Branch Merge: Use Git to merge your branches completely
3. Discard DaVinci Project Merge: Discard any changes that affect your DaVinci Project
4. DaVinci Project Merge with Comparison Mode
5. Test the Merge Result (CI/CT)
6. Merge Completion
7. Push your changes

© 2022 Vector Informatik GmbH

3.2 Git & DaVinci Merge Steps

The following is a merge example with DaVinci Configurator. However, the same principles can be used for DaVinci Developer. Below you will find the extended explanations of each step of the Merge.

3.2.1 Getting local copies of files to be merged

Git is commonly used to merge more than just the DaVinci Project. Before any merging is done, it is recommended to already have local copies of the DaVinci Projects that are needed for the merge. For example: MINE (local working directory), BASE and OTHER (local copies) projects.

Consider the following:

> You can only check out one branch at a time.

> DaVinci Configurator requires at least a second local existing DPA project to do a merge (for our case three, due to the three-way merge).

Option 1:

Checkout the needed branch (For example, OTHER) and locally copy the DaVinci Project from it (For an example, see the first step of Chapter 4 DaVinci Project Merge with Comparison Mode).

Option 2:

Have multiple repositories, where a separate branch is checked out in each.

As an example, you could have two repositories, and checkout where the BASE and OTHER projects are. Figure 3- shows it using “clone” to merge.

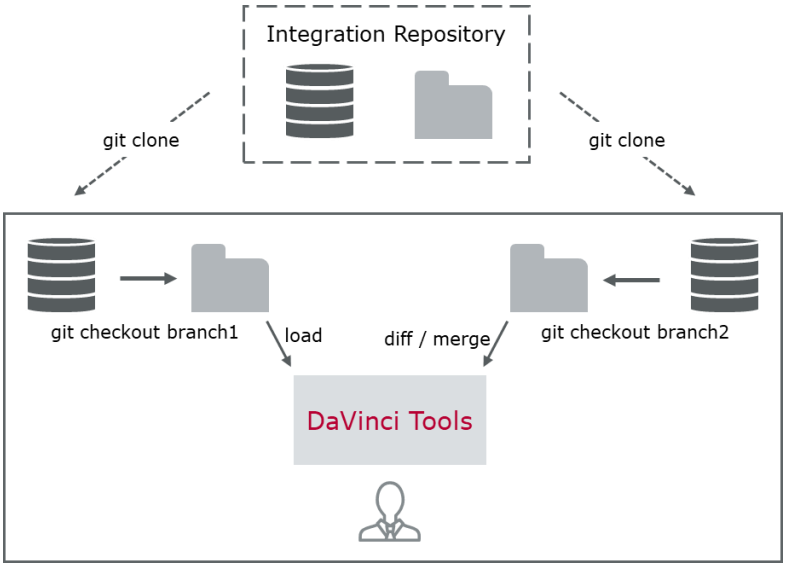


Figure 3-2 Two full projects are needed to make a DaVinci Configurator Merge



Note
Instead of making a clone of a repository, consider the `worktree` command, which can be used to checkout other commits.

3.2.2 Git Branch Merge

Now you have local copies of your projects you want to merge (BASE, OTHER and MINE), assuming that we are merging more than just the DaVinci Project we can use Git´s textual merge, to merge the two branches we are interested in.

The purpose of this step is to merge everything that has nothing to do with the DaVinci ARXMLs and DPA files. Since there is no way to selectively merge, we can simply merge “everything” as a first step. Any conflicts rising from the DaVinci Project Git Merge, can be discarded, as they will be discarded on the next step anyway.

3.2.3 Discard DaVinci Project Merge

Now that the whole project is merged through Git, it is time to discard the merging done to the DaVinci Project. What exactly should be discarded? The following:

- ARXML files
- DPA file
- DCF file
- DVG files

3.2.4 DaVinci Project Merge with Comparison Mode

Now that the DaVinci Project has been taking to its original unmerged structure, we can start with the DaVinci Project Merge through Comparison Mode.



Example Scenario
We are trying to merge BLUE branch shown in Figure 3-1 into the main/develop branch.

A developer already added its changes to the develop branch, so you can´t simply add your changes. You need to do a DaVinci Project Merge.

1. Based on Chapter 3.2.1, you should already have the copies of your DaVinci Projects (OTHER and BASE). If not yet done, this would be an example of copying your OTHER project from the **develop** branch to a new directory.

   Example: LightControl_OTHERProject checkout

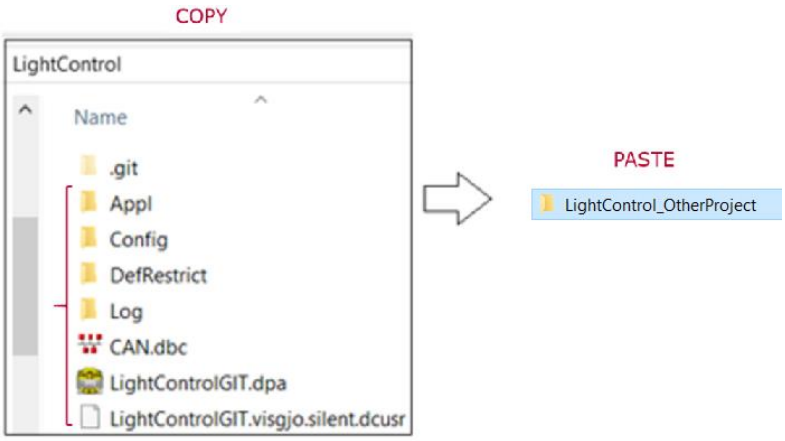


Figure 3-3 Copy the Other project from the develop branch to a new directory

2. Switch back to your feature branch to use it as MINE, since we will apply the changes in that project.
3. From the repository directory, (currently on feature branch) open the CFG5 Project and click on **Start Comparison Mode**. This project will be taking the position of role of MINE.

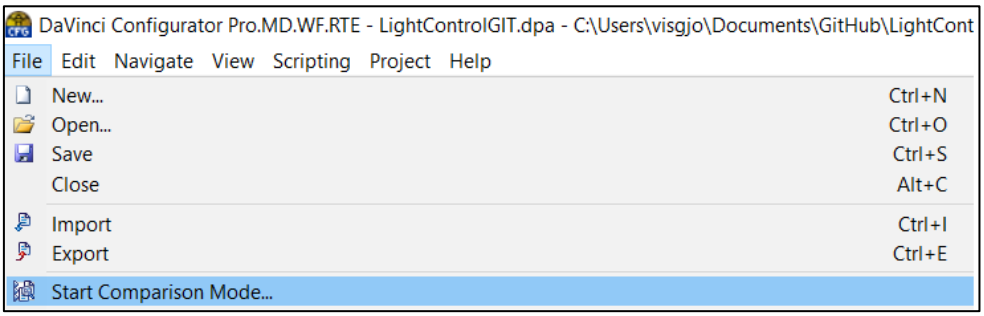


Figure 3-4 Start Comparison Mode on develop branch

4. In the window **Compare and Merge Project**, select the project on LightControl_OtherProject as **OTHER.** Also select your **BASE** project under Base.

   Then click **[NEXT]**.

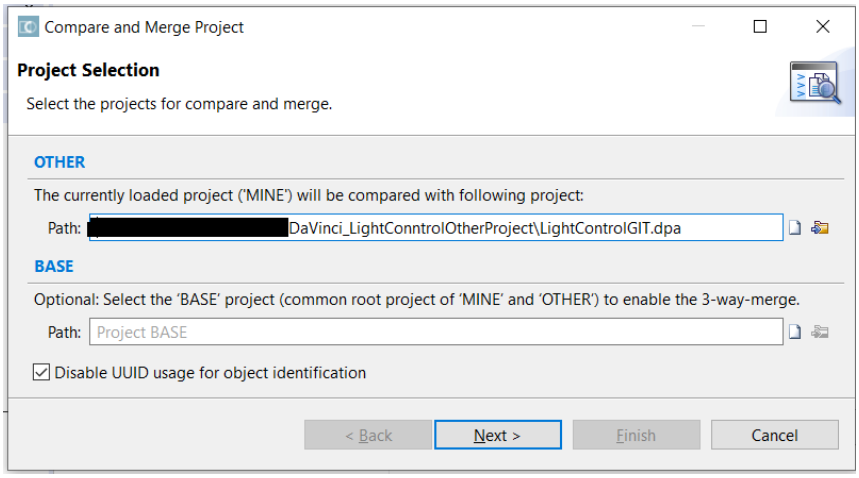


Figure 3-5 Select the Project to be merged

5. In case there might be changes on the Developer Workspace, click on **Merge DaVinci Developer Workspace**.

This opens DaVinci Developer and shows any differences between the projects.

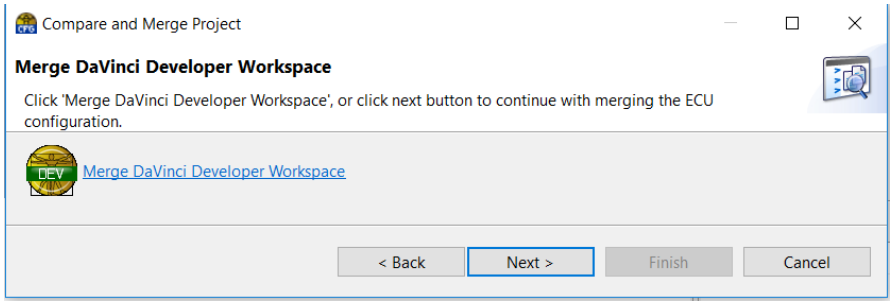


Figure 3-6 Merge DaVinci Developer option

6. DaVinci Developer Differences Pop Out in 3 categories:
   - Added Elements
   - Modified Elements
   - Removed Elements

   The three-way merge allows you to auto merge all non-conflicting differences. In addition, you can manually select which changes to **Apply** or which ones to **Ignore**. In this case, only added elements are visible.

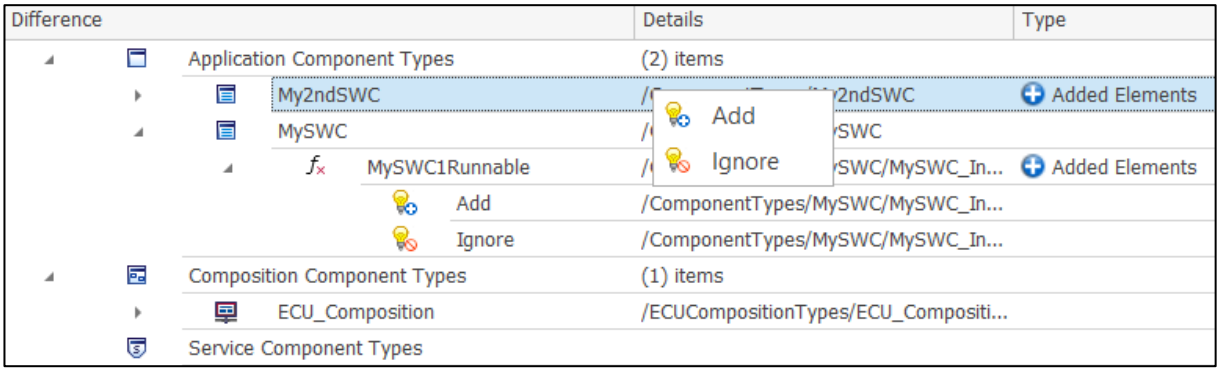


Figure 3-7 “My2ndSwc” exists only on the develop branch. Select “Add” to add it or “Ignore” to leave it out of the project

7. After changes, exit DaVinci Developer, and click **[Next]** on the **Compare and Merge Project** to continue with merging the EcuC Configuration.
8. Now select which modules to compare. Select the desired options and click **[Next]**.

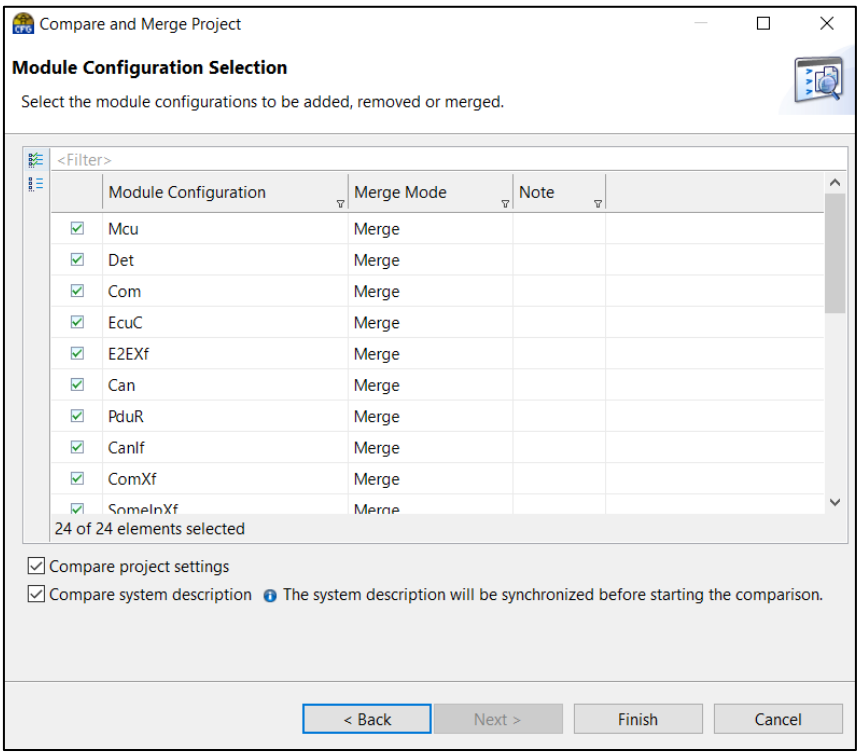


Figure 3-8 Decision on which EcuC modules to compare

9. The **Differences** tab now shows the differences between the two projects.

   As with DaVinci Developer, the auto-merge button can merge all non-conflicting differences. For conflicts a manual decision is required.

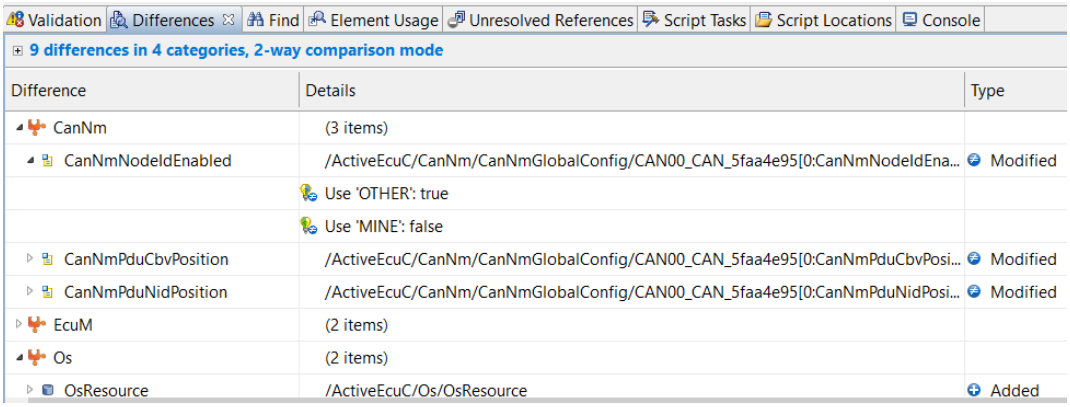


Figure 3-9 By CFG5 select which modifications to Add, Remove or Keep

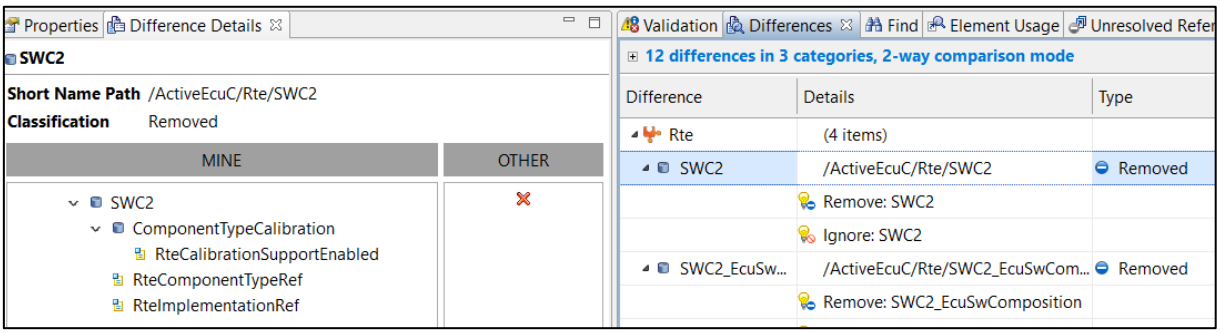


Figure 3-10 A selection also triggers the Difference Details window (left), which may give better visualization

10. After being satisfied with the changes, save the project.

3.2.5 Test your Merge (CI/CT)

Before committing and or pushing your results, it is highly recommended to test the merge result first. CI/CT pipelines could be used to confirm that the resulting merge is valid. Alternatively, you could simply open the merged project with DaVinci Configurator and/or DaVinci Developer, to confirm that the result was satisfactory and there are no unexpected errors.

3.2.6 Merge Completion

After the DaVinci Project Merge procedure, by saving the working directory on the feature branch, it will now contain all your changes. However, the changes are not committed yet.

For organizational purposes (this step is optional), the two used branches can be marked as merged in Git with the following command without committing anything:

```
git merge -s ours <branch_to_merge>
```

This completely discards the develop branch and takes the current state of the **feature** branch as the result of the merge (which it is, since half was merged through Git, and the DaVinci Project was merged through DaVinci Configurator).

3.2.7 Push your changes

Now that the feature branch has the latest version and the validity has been confirmed, you can push the changes to the remote server. Since the feature branch now has merge result, it can also be pushed directly to the develop branch.

© 2022 Vector Informatik GmbH

4 Glossary and Abbreviations

4.1 Glossary

Term	Description
Push	Use to update a branch on the server repository with the user´s own updated branch. The branch may be in another repository, as when updating the develop branch on a server repository.
Pull/Fetch	To take the latest version of a branch. Example: “Pull” the latest version of the develop branch from the server repository.
Commit	Used to “Save” changes into the currently selected branch.
Merge	The act of combining two separate files or projects.
Clone	Refers to the act of copying a server´s repository into the user´s own local computer.
Server Repository	Refers to the repository where all the developers push their changes, making it the one branch where all the updates are pushed into. In the same way, it is open for everyone to fetch or pull the latest version of.

Table 5-1 Glossary

4.2 Abbreviations

Abbreviation	Description
VCS	Version Control System: A system for recording and managing changes to documents or files.
SVN	Subversion: A concrete centralized VCS, which was very popular until a few years ago, but can still be found every now and then. Lately it is being replaced more and more by distributed systems like Git.
CVS	Concurrent Versions System: The unofficial predecessor of SVN is practically irrelevant today, as the system is no longer developed or maintained.
CI/CT	Continuous Integration / Continuous Testing. Pipelines used to confirm the validity of a project.

Table 5-2 Abbreviations

5 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2022 Vector Informatik GmbH